Perception Docker Guide

The Docker Compose file defines a **containerized setup** for an autonomous driving perception and control pipeline. It uses **Docker** with ROS Melodic, Conda environments, and GPU acceleration to run different perception and control modules in isolated services. Here's a breakdown:

Overall Structure

- **Version:** Uses Docker Compose v3.3 syntax.
- **Services:** Each ROS node/module runs as its own container.
- **Images & Build:**
  - A base image `parimiharsha/perception_image` is built once using `perception_image_builder`.
  - All other containers reuse this image.
- **GPU & ROS Integration:**
  - `runtime: nvidia` enables GPU acceleration for modules like YOLO, SAM2, and SphereFormer.
  - `network_mode: "host"` allows containers to communicate with the ROS master on the host system.
- **Volumes:** Mounts source code and Conda environments from host → container for development flexibility.

Key Services

1. **perception_image_builder**
   - **Role:** Build-only container for compiling the ROS workspace (`catkin_make`).
   - **Purpose:** Ensures all source code and dependencies are ready before launching perception modules.

---

1. **yolo_node**

- **Role:** Runs **YOLOv9 object detection** ROS package.
- **Key Steps:**
  - Activates the `yolov9` Conda environment.
  - Sources ROS setup and workspace files.
  - Launches `yolov9_ros.launch`.

---

1. **sam_node**
   - **Role:** Runs **SAM2 segmentation** ROS package.
   - **Key Steps:** Similar to YOLO but uses the `sam2` Conda environment.

---

1. **clrernet_node**
   - **Role:** Runs **ClreNet ROS** for lane detection or perception tasks.
   - **Key Steps:**
     - Adds additional paths to `PYTHONPATH` for ClreNet source code.
     - Activates `clrernet` environment and launches `clrernet_ros.launch`.

---

1. **transform_node**
   - **Role:** Runs **transform.py** script for LiDAR 2D projection transformation.
   - **Key Steps:**
     - Activates `yolov9` environment.
     - Executes the Python transform script directly (not a ROS launch file).

---

1. **sphereformer_node**
   - **Role:** Runs **SphereFormer LiDAR segmentation**.
   - **Key Steps:**
     - Activates `sphereformer` environment.
     - Sets CUDA and Conda library paths for GPU-based processing.
     - Launches `sphereformer_ros.launch`.

---

1. **controller_node**
    - **Role:** Runs the **AVA_Control** ROS package for vehicle control.
    - **Key Steps:**
        - Installs required ROS dependencies (`raptor-dbw-msgs`).
        - Compiles the workspace.
        - Activates `newcontrol` environment for Python2-based control scripts.
        - Installs legacy `pip2` and SciPy dependencies.
        - Launches `ctrl6.launch` for the controller.

Workflow

1. **Build phase:**

   `perception_image_builder` builds the workspace once using source code.

2. **Perception modules:**

   `yolo_node`, `sam_node`, `clrernet_node`, `sphereformer_node`, `transform_node` run in separate containers to handle different perception tasks.

3. **Control module:**

   `controller_node` receives perception outputs and sends control commands to the vehicle.

4. **Runtime:**

   All containers share the same ROS master, GPU resources, and code base via mounted volumes.

Docker Compose Cheat Sheet

Starting & Running Services

- **Start a specific service:**

```
docker compose up <service_name>
```

Example:

```
docker compose up yolo_node
```

- **Start service with a profile:**

  ```
  docker compose --profile <profile_name> up <service_name>
  ```

  Example:

  ```
  docker compose --profile builder up perception_image_builder
  ```

- **Run a service with interactive terminal:**

  ```
  docker compose run --rm --service-ports <service_name> bash
  ```

  Example:

  ```
  docker compose run --rm --service-ports yolo_node bash
  ```

Container Lifecycle

- **Create & start all services:**

  ```
  docker compose up
  ```

- **Start services without creating:**

  ```
  docker compose start
  ```

- **Stop services:**

  ```
  docker compose stop
  ```

- **Remove containers, networks, volumes:**

```
docker compose down
```

- **Specify a custom Compose file:**

```
docker compose -f custom-docker-compose.yml up
```

Logs & Debugging

- **Check logs of a service:**

```
docker compose logs <service_name>
```

- **Follow logs in real-time:**

```
docker compose logs -f <service_name>
```

Building & Rebuilding Images

- **Build a specific service image:**

```
docker compose build <service_name>
```

Example:

```
docker compose build perception_image_builder
```

Accessing Containers

- **Access a running container:**

```
docker compose exec <service_name> bas
```

Example:

```
docker compose exec clrernet_node bash
```